

Pontifícia Universidade Católica de Minas Gerais
Bacharelado em Ciência de Dados e Inteligência Artificial – Campus Lourdes
Programação Modular
Semestre 2/2026
Atividade Pontuada em Laboratório 1 – Valor: 3 pontos

Esta é uma atividade pontuada em laboratório da disciplina Programação Modular, no valor de 3 pontos. Leia as instruções com atenção antes de começar a atividade. Chame o professor em caso de dúvidas.

ATENÇÃO: Nesta atividade não é permitido o uso de agentes de código nem consulta a material externo. Você pode debater a solução com seus colegas e com o professor, consultar o material da disciplina e o código das aulas anteriores.

PARTE 1: PREPARAÇÃO DA ATIVIDADE

- Vá ao repositório em https://github.com/ProgramacaoModular/lab_AP1_2026-2_M2 e faça um fork para seu usuário do GitHub;
- Crie um clone do seu repositório no computador do laboratório e abra este clone na IDE;
- Em seguida, crie uma nova branch a partir da main, com nome Aula2109;
- Habilite os testes do projeto com JUnit Jupiter;
- Chame o professor, que irá desconectar o computador da rede.

PARTE 2: TAREFAS DE PROGRAMAÇÃO

A Xulamb's Mobility opera estações de aluguel de patinetes elétricos.

Cada patinete é identificado por um código de 4 dígitos e possui carga de bateria (0 a 100). O estado operacional é:

- indisponível — bateria menor que 20%;
- disponível — bateria entre 20% e 100% (inclusive) e o patinete não está em uso;
- em_uso — há um aluguel ativo.

O patinete só inicia aluguel se estiver disponível. Ao encerrar, deixa de estar em uso (e pode voltar a indisponível se a bateria estiver baixa).

Uma estação agrega vários patinetes (eles existem independentemente da estação). A estação deve:

1. adicionar um patinete à sua frota (não adicionar o mesmo código duas vezes);
2. `liberarDisponivel(codigo)` — localizar o patinete pelo código; se estiver disponível, iniciar o aluguel e retornar `true`; caso contrário, `false`;
3. `informarTotalDisponiveis` — quantidade de patinetes no estado disponível;
4. calcular o aproveitamento da frota: $\text{proporção em_uso} / (\text{disponível} + \text{em_uso})$. Se não houver nenhum patinete disponível nem em uso (todos indisponíveis ou frota vazia), o aproveitamento é 0. Se houver patinetes em uso e nenhum disponível, o aproveitamento é “máximo”

(Double.MAX_VALUE);

5. verificar se esta estação está na frente de outra: primeiro pelo aproveitamento da frota; em empate, pela quantidade de patinetes disponíveis (mais disponíveis fica na frente); se empatar nos dois critérios, não há frente (retorna false);

6. produzir um resumo em uma linha: código da estação, total de patinetes, total disponíveis, total em uso e aproveitamento em porcentagem (quando for MAX_VALUE, exibir MAX).

A classe Patinete e seu teste já estão implementados. Na classe Estacao, os métodos adicionar, totalDisponiveis, totalEmUso e resumo também já estão prontos, assim como alguns testes em EstacaoTest, que servem de exemplo. Suas tarefas estão marcadas com //TODO nas classes Estacao e EstacaoTest.

TAREFAS

- Tarefa 1: implementar liberarDisponivel em Estacao;
- Tarefa 2: implementar aproveitamentoFrota em Estacao;
- Tarefa 3: implementar estaNaFrenteDe em Estacao;
- Tarefa 4: completar o teste deveCalcularAproveitamentoFrota em EstacaoTest;
- Tarefa 5: completar o teste deveCompararEstacoes em EstacaoTest;
- Ao terminar, chame o professor para reconectar o computador na rede;
- Realize um commit com mensagem Final da atividade e publique sua branch;
- Confirme a realização do exercício enviando o endereço do seu repositório na tarefa do Canvas.

ATENÇÃO: Para estas tarefas você pode modificar quaisquer arquivos/classes, de acordo com o que julgar necessário.

PONTUAÇÃO

- 0,5 ponto para os procedimentos de fork, clone, commit e push;
- 2,5 pontos para as tarefas de programação e de teste: 0,5 ponto por tarefa (Tarefas 1 a 5). Os testes das Tarefas 4 e 5 também são avaliados: devem chamar o método e verificar o resultado esperado com assertivas.